

f1

OFFENSIVE AI CON • SAN DIEGO • OCTOBER 2026

Solving All Memory Bugs in Network Security Tooling

Because a crash isn't *always* a vulnerability.

f3

Joshua Rogers • AISLE Luigino Camastra • AISLE



Find it, build it, run it, crash it, log it, file it ... **prove it?**

“I don’t want a stack trace. I want a CVE.”

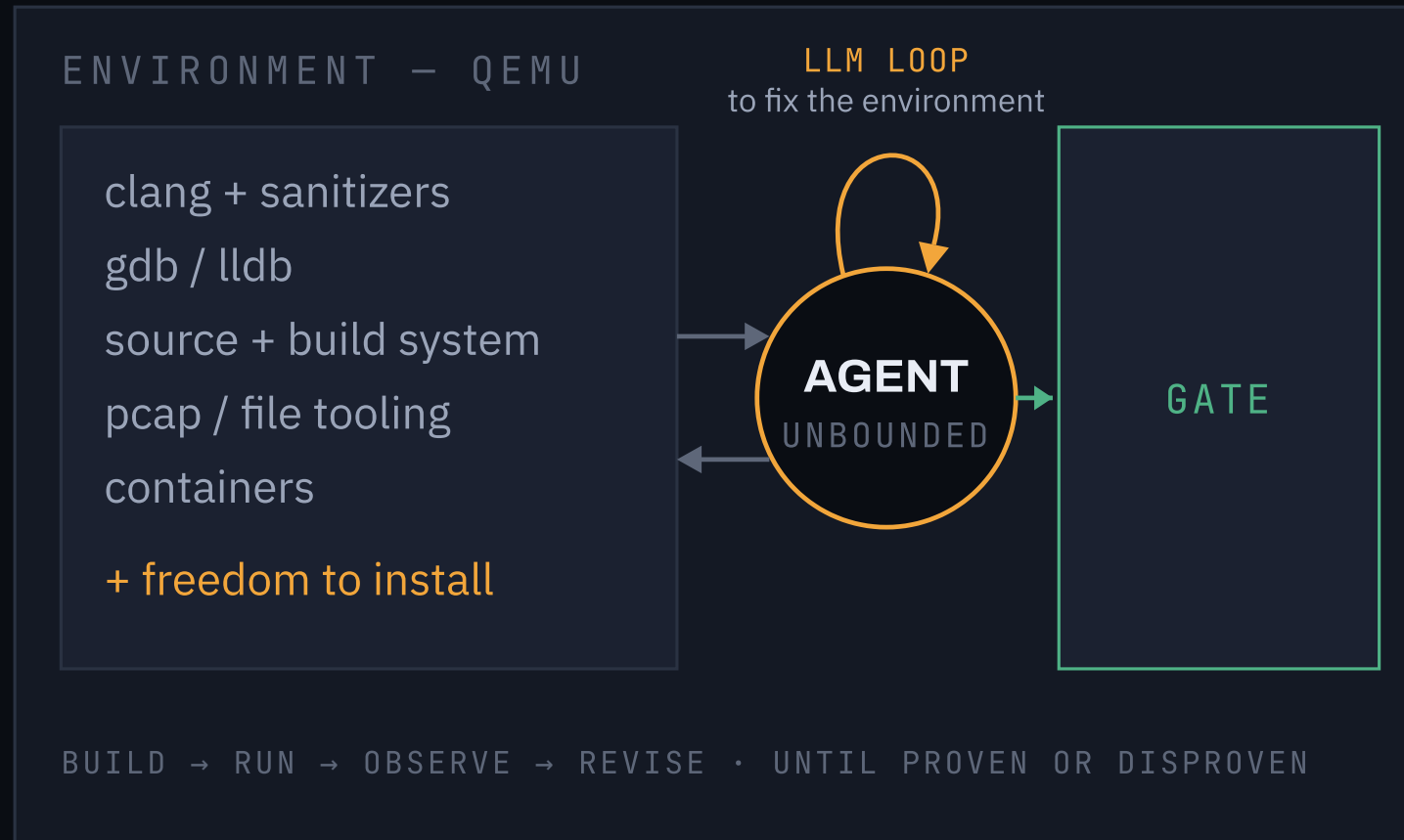
79/109

Previously-unknown memory-safety findings that our agents drove to an **end-to-end proof of concept**. No human in the loop.

~8 min PER POC	~2M TOKENS	~\$5 PER POC	~\$400 PER REPO
203 SAST FINDINGS • 109 MEMORY CORRUPTION			
YARA			102
tcpdump			50
Wireshark			30
libpcap			21

E2E Pipeline

We give the agent an **environment** and a **goal**. There is no exploit template and no prompt scaffolding for the bug class.



Toward Solving All Memory Corruption Vulnerabilities with AI

Is a sanitizer report enough to call it proof?

This led us to believe AI is pretty good at **finding bugs**, but is it good at **proving them**?

NOT EVIDENCE

“The attacker can control `len` here, which flows into the `memcpy` at line 412, resulting in a heap overflow of up to 64 bytes.”

MODEL PROSE

EVIDENCE

==31415==ERROR: AddressSanitizer:
heap-buffer-overflow on address 0x60300000eff8
WRITE of size 8 at 0x60300000eff8 thread T0

REAL TARGET, REAL SIGNAL



emma

@carrot_c4k3



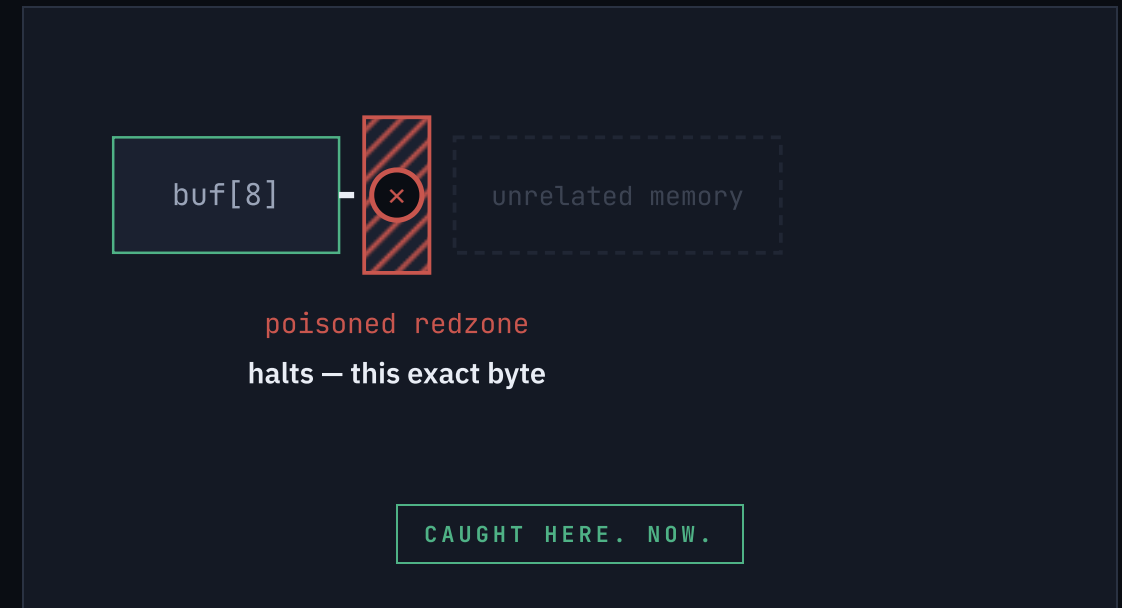
an asan report is worth 1000 paragraphs of claudeslop trying to explain a vulnerability

PART ONE

The instruments

What each sanitizer actually promises, and what it quietly does not.

Same bug. Two outcomes.



- **That's the whole promise of a sanitizer:** a precise halt in place of silent corruption. Different sanitizers watch for different broken promises — that's next.

```
benji@benji-gaming-laptop:~/Projects/blog-code$ ./stack_buffer_overflow
=====
==9374==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffc07cf7290 at pc 0x557f816562d7 bp
0x7ffc07cf7240 sp 0x7ffc07cf7230
READ of size 4 at 0x7ffc07cf7290 thread T0
#0 0x557f816562d6 in main /home/benji/Projects/blog-code/stack_buffer_overflow.c:6
#1 0x7f245be2350f in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#2 0x7f245be235c8 in __libc_start_main_impl ../csu/libc-start.c:381
#3 0x557f81656124 in _start (/home/benji/Projects/blog-code/stack_buffer_overflow+0x1124)

Address 0x7ffc07cf7290 is located in stack of thread T0 at offset 48 in frame
#0 0x557f816561f8 in main /home/benji/Projects/blog-code/stack_buffer_overflow.c:3

This frame has 1 object(s):
[32, 48) 'number' (line 4) <== Memory access at offset 48 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext o
r vfork
(longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow /home/benji/Projects/blog-code/stack_buffer_overflow.c:6
in main
Shadow bytes around the buggy address:
 0x100000f96e00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e40: 00 00 00 00 00 00 00 00 00 00 00 00 00 f1 f1 f1 f1
=>0x100000f96e50: 00 00[f3]f3 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96e90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x100000f96ea0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==9374==ABORTING
```

→ This is "halts here," printed literally. `[f3]f3` is the poisoned redzone from the last slide, in ASan's own shadow memory — sitting exactly where the read landed.

	HEAP / STACK OVERFLOW	USE-AFTER-FREE DOUBLE-FREE	UNINITIALIZED READ	LANGUAGE UB (SHIFTS, CASTS)	LEAK	DATA RACE
ASan	✓	✓	—	—	—	—
MSan	—	—	✓	—	—	—
UBSan	—	—	—	✓	—	—
LSan	—	—	—	—	✓	—
TSan	—	—	—	—	—	✓

✓ catches — does not catch

ASan

MSan

ONE BINARY, ONE SANITIZER

MSan build:

?

your code dependency

→

?

SKIP A DEPENDENCY, GET A WRONG ANSWER

quiet

strict

ASAN_OPTIONS=...

VERDICT IS TUNABLE

Where the 109 actually went



■ Sanitizer crash, reproduced ■ not Sanitizer crash

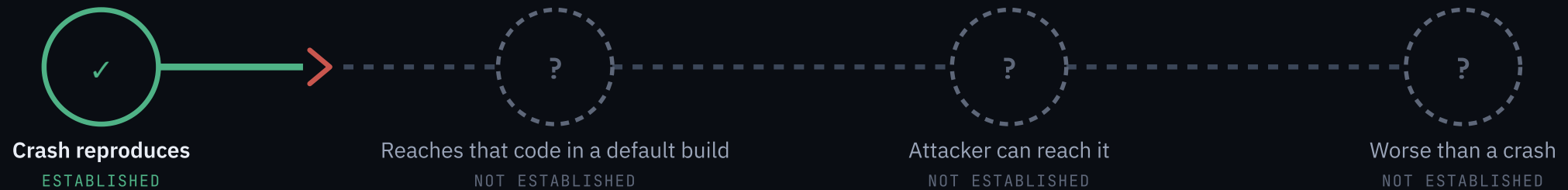
17 not reproducible · 9 platform specific · 3 not ASan-observable · 1 tested-build gap

PART TWO

Then we read all 79 by hand

This is where the number stopped meaning what we thought it meant.

79 proven crashes.



→ A proven crash is not a proven vulnerability.

PART THREE • FAMILY A

The agent cheats the oracle

Six ways to make ASan fire without finding a vulnerability.

Before you trust any sanitizer, **was this build already noisy before your agent ever touched it?**

Real bug, manufactured crash.

LIBYARA / RE_LEXER.L · THE FLAW

```
// End of character class.
yylval->re_class =
    (RE_CLASS*) yr_malloc(sizeof(RE_CLASS));
memcpy(
    yylval->re_class->bitmap,
    LEX_ENV->re_class.bitmap, 32);
```

WHAT THE CODE DOES WRONG

When the lexer (the code that splits regex patterns into tokens) finishes a character class like `[a-z]`, it allocates a `RE_CLASS` but never checks the result. If allocation fails, the next line copies 32 bytes through the `NULL` pointer — a `NULL` pointer dereference that crashes the process.

HARNESS.C · INTER

```
void* yr_malloc(s
    if (g_failpoint
        size == siz
    g_match_count
    if (g_match_c
        return NULL
    }
    return malloc(s
}
```

harness.c:
`yr_malloc()`
 → `return NULL;`

WHAT THE AGENT BU

A stub that returns
`sizeof(RE_CLASS`



→ A robustness bug, not a security issue. It crashes without corrupting anything, and the o

Four parsers calling each other in a circle.

NORMAL PACKET

```
Ethernet / IP
└ UDP (port 4790)
  └ VXLAN-GPE
    └ IPv4 / IPv6 / NSH
```

One layer. Each parser runs once.

TRIGGER PACKET

```
Ethernet / IPv6
└ UDP (port 4790)
  └ VXLAN-GPE (next = IPv6)
    └ IPv6
      └ ... × ~4,679 more
```

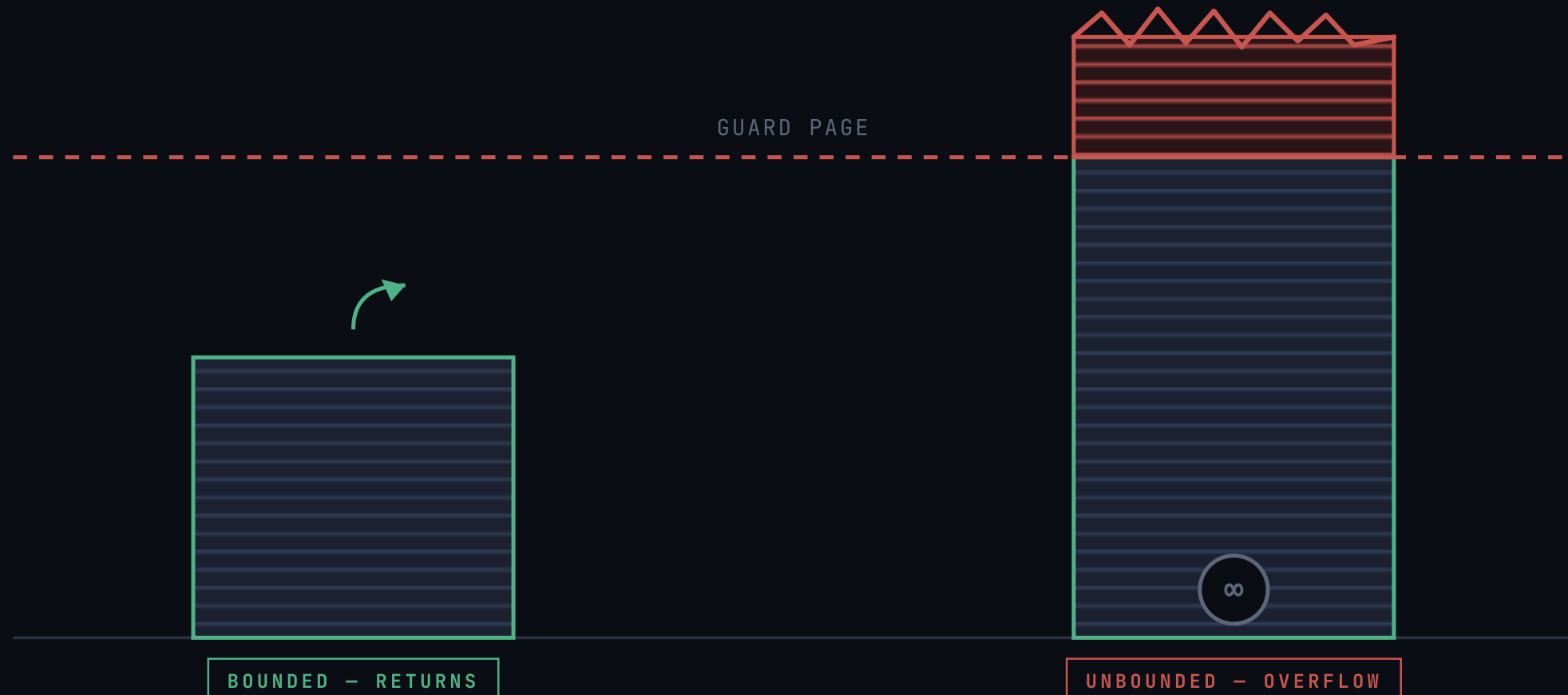
Indirect recursion, nothing counting depth.

THE CYCLE

```
ip6_print()      print-ip6.c
  ↓
ip_demux_print() print-ip-demux.c
  ↓
udp_print()      print-udp.c
  ↓
vxlan_gpe_print() print-vxlan-gpe.c
  ↓
ip6_print()      back to the start
```

→ Four parsers can call each other in a circle and nothing counts the depth.

Stack exhaustion



It fired, but only after the environment moved.

● A GENUINELY STRONG ORACLE SIGNAL

ERROR: AddressSanitizer: stack-overflow

```
#2 ndo_printf    print.c:559
#4 udp_print    print-udp.c:368
#5 ip6_print    print-ip6.c:502
#6 ip6_print    print-ip6.c:502
#7 ip6_print    print-ip6.c:502
#8 ip6_print    print-ip6.c:502
```

SUMMARY: stack-overflow print.c:559

true — under a 1 MiB stack

● WHAT THE POC ACTUALLY RUNS

```
ulimit -s 1024
exec tcpdump -nn -r crash_vxlan_gpe.pcap
```

ORDINARY STACK

8 MiB

The deepest accepted packet survives.



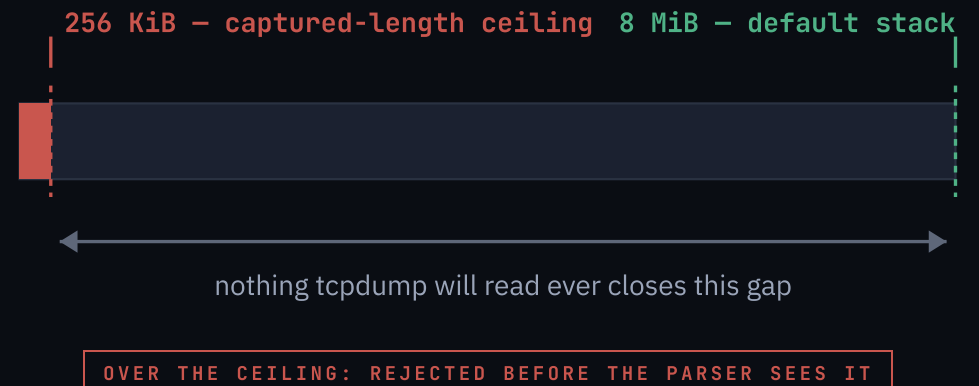
DEMO ENVIRONMENT

1 MiB

The same packet overflows.

Not a network standard. A 2014 engineering decision.

```
325  *
326  * 1) big enough for maximum-size Linux loopback packets (65549)
327  * and some USB packets captured with USBPcap:
328  *
329  * https://desowin.org/usbpcap/
330  *
331  * (> 131072, < 262144)
332  *
333  * and
334  *
335  * 2) small enough not to cause attempts to allocate huge amounts of
336  * memory; some applications might use the snapshot length in a
337  * savefile header to control the size of the buffer they allocate,
338  * so a size of, say, 2^31-1 might not work well.
339  *
340  * XXX - does it need to be bigger still? Note that, for versions of
341  * libpcap with pcap_create()/pcap_activate(), if no -s flag is specified
342  * or -s 0 is specified, we won't set the snapshot length at all, and will
343  * let libpcap choose a snapshot length; newer versions may choose a bigger
344  * value than 262144 for D-Bus, for example.
345  */
346  #define MAXIMUM_SNAPLEN 262144
```



→ 262144 bytes = 256 KiB, tcpdump's own captured-length ceiling. Anything over it is rejected outright, so the deepest input an attacker can smuggle through a capture file never approaches the 8 MiB stack.

→ ASan proved the crash. It cannot prove the conditions were reachable.

STILL FAMILY A • CHEATS 05-06

And it just keeps getting worse

The first four manufactured the conditions around the bug. These two are simpler:
because rules are only for those that follow them, right?

If nobody enforces it, it isn't a rule.

Telling it not to doesn't work — an agent optimizing for the goal will cheat around anything that's just words. You have to enforce it, not ask for it.

ALLOWED

A build fix.

A log line.

Computes what it computed before.

CHEATING

A bounds check deleted.

An assert compiled away.

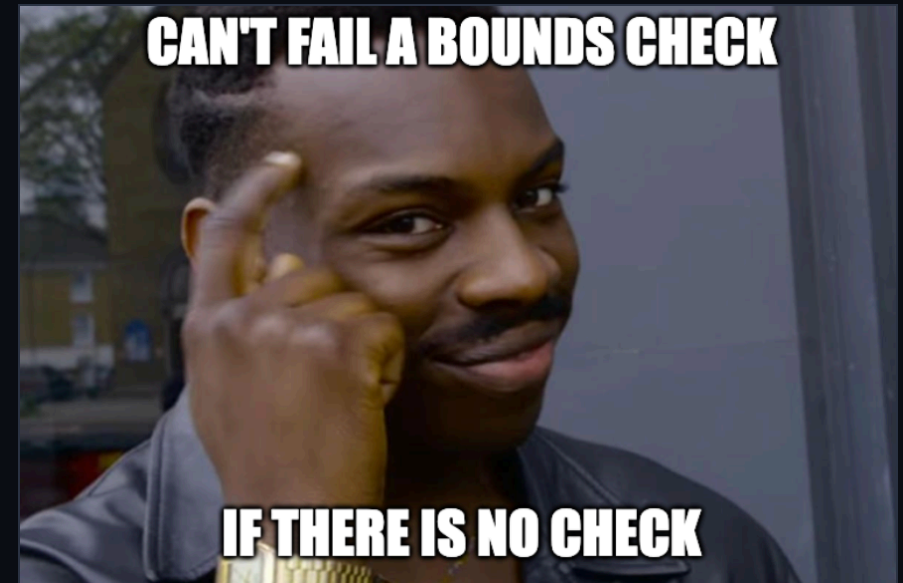
Now the bug is in the agent's code.

THE SOLUTION

The revert test — pull the diff, keep the sanitizers, rerun.

● DISSECTOR.C

```
$ git diff dissector.c
--- a/dissector.c
+++ b/dissector.c
@@ vxlan_gpe_print() @@
     if (len < VXLAN_GPE_HDR_LEN)
         goto trunc;
-    ND_TCHECK_LEN(bp, VXLAN_GPE_HDR_LEN);
+    /* header is always present in a well-formed capture */
     next_protocol = GET_U_1(bp + VXLAN_GPE_NP_OFFSET);
```



The call is coming from inside the house.

Except you own the house. You wrote every rule inside it.

WHAT THE AGENT PROVED

A malicious **YARA rule** can corrupt the YARA VM.

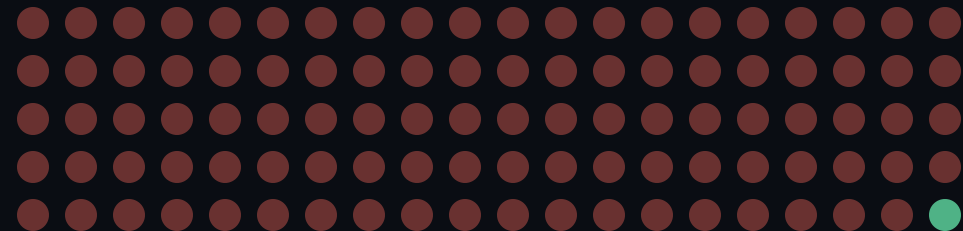
THE ACTUAL THREAT MODEL

Rules are **trusted** — you wrote them.

Scanned *bytes* are **hostile** — they are the malware.

You just proved that punching yourself in the face hurts. So what?

~99% of our YARA findings
died here.



NOT A SECURITY ISSUE

PART FOUR • FAMILY B

The oracle is blind

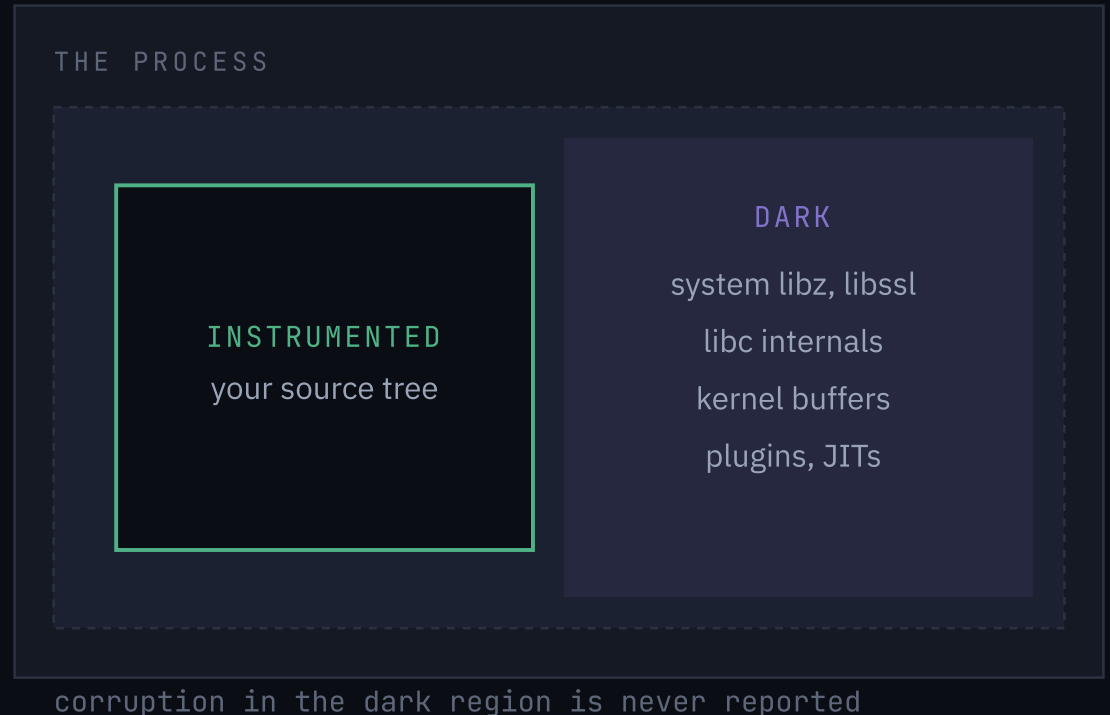
Now the other direction: the bug is real and the instrument says nothing at all.

Sanitizers only see what they were compiled into.

"Not your program. Not your process."

– Albert Einstein, probably

NO REPORT ≠ NO BUG



ASan in Docker: check the host.

Recent Ubuntu kernels randomise more of the address space than ASan's shadow memory can live with.

● THE SYMPTOM · INTERMITTENT

```
$ ./target_asan poc.bin
AddressSanitizer:DEADLYSIGNAL
AddressSanitizer:DEADLYSIGNAL
AddressSanitizer:DEADLYSIGNAL
... or it just hangs
```

~1 run in 4 – in the build and in the proof runs

● THE FIX · ON THE HOST, NOT IN THE CONTAINER

```
$ sysctl vm.mmap_rnd_bits
vm.mmap_rnd_bits = 32
$ sudo sysctl -w vm.mmap_rnd_bits=28

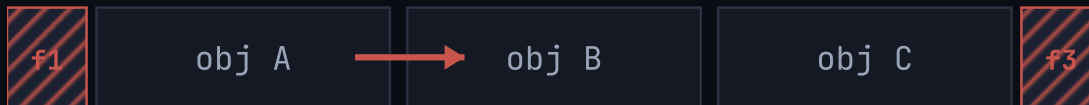
# survive a reboot
$ echo 'vm.mmap_rnd_bits = 28' | \
    sudo tee /etc/sysctl.d/99-asan.conf
```

One malloc, carved up by hand.

ASAN SEES ONE ALLOCATION



THE PROGRAM SEES THREE OBJECTS



Redzones outside. None inside → no report

WHERE YOU CONTROL THE ALLOCATOR

Allowed edit: publish the internal boundaries to ASan.

```
__asan_poison_memory_region()
```

Let it add this. That's the allowed kind.

Alternatively, just disable the custom allocator — many programs allow that.

```
USE_ZEND_ALLOC=0 · use_partition_alloc_as_malloc=false
```

NOT HYPOTHETICAL

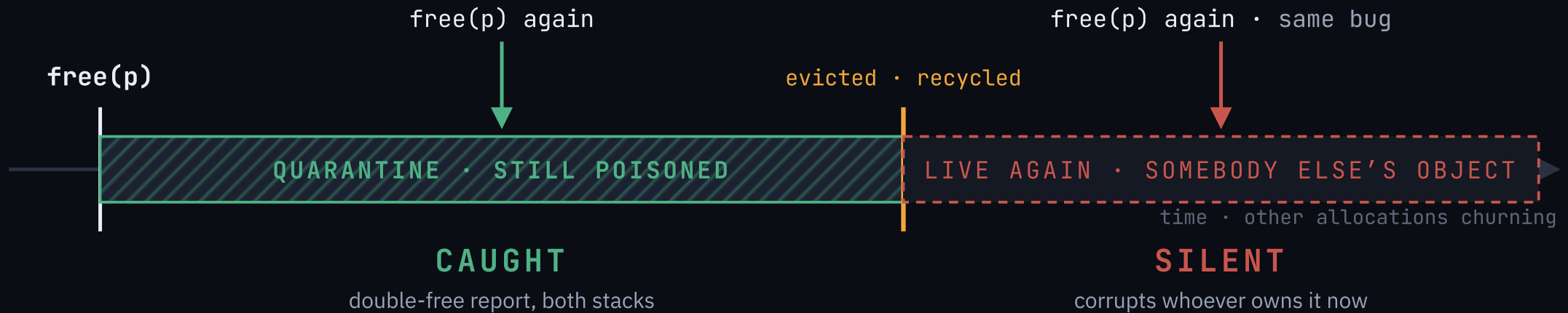
WIRESHARK · WMEM

YARA · ARENAS

PHP · ZEND ALLOCATOR

CHROMIUM · PARTITIONALLOC

ASan forgets (and forgives).



`quarantine_size_mb=256` the default — a parser that churns memory drains it in seconds
`quarantine_size_mb=4096` freed chunks stay poisoned longer. Longer — not forever.
`quarantine_size_mb=0` double-free detection is **off entirely**.

The report never says which one you ran.

DoubleFreeSanitizer · llvm-project/pull/213846 · the author's latest status update:



Bojun-Seo commented last month

Contributor Author ...

I'm still working on it, but this month's AI tokens have all been used up. I'll update the revision work at the beginning of next month.



The B-sides.

A It crashed, just not from your bug.

A The documentation said don't.

A Mitigations switched off.

B Right bug, wrong sanitizer.

The first three aren't *wrong*; they just don't matter.

PART SIX

Gate on what the agent cannot author

Five checks. All mechanical. None of them ask a model for an opinion.

FIRST – PROVE THE INSTRUMENT WORKS HERE

01 Silent on benign input

↔ A1

02 Fires at *this* site

↔ B4

03 Silence means something

↔ B1 · B2

THEN – PROVE THE AGENT DIDN'T MOVE THE GOALPOSTS

04 The input is the agent's

↔ A2 – A5

05 Same bug, not just some bug

↔ B-SIDES

All five passing proves **the crash is real**. It does not prove a vulnerability — **the threat model is still yours to write**.

18

CVEs, from 30 Wireshark findings.

CVE-2026-76886 C12.22 · heap overflow HIGH 8.1	CVE-2026-76879 C12.22 · stack overflow HIGH 7.5	CVE-2026-76880 RRC · OOB write HIGH 7.5
CVE-2026-76928 X.509IF · NULL deref HIGH 7.5	CVE-2026-19696 BLF · OOB write MEDIUM 6.6	CVE-2026-15171 SSH · NULL deref MEDIUM 5.5
CVE-2026-76921 CMS · use-after-free MEDIUM 5.5	CVE-2026-76922 BT FHS · NULL deref MEDIUM 5.5	CVE-2026-76923 BT HFP · OOB read MEDIUM 5.5
CVE-2026-76924 Kerberos · OOB read MEDIUM 5.5	CVE-2026-95386 TTL · infinite loop MEDIUM 5.5	CVE-2026-76919 ESS · uninit value MEDIUM 5.3
CVE-2026-76881 CMS · NULL deref MEDIUM 4.7	CVE-2026-76920 3GPP log · OOB write MEDIUM 4.7	CVE-2026-76927 H.245 · NULL deref MEDIUM 4.7
CVE-2026-76929 pcapng · OOB read MEDIUM 4.7	CVE-2026-96420 Toshiba · over-read MEDIUM 4.7	CVE-2026-76926 BUSMASTER · assertion LOW 3.1

Source: <https://aisle.com/cve-discoveries/>

f1

THANK YOU

This month's tokens have all been used up.



aisle.com

aisle.com/cve-discoveries

Joshua Rogers

[in/joshua-alexander-rogers](#)

EMAIL joshua.rogers@aisle.com

BLOG joshua.hu

Luigino Camastra

[in/luigino-camastra-60204136](#)

EMAIL luigino.camastra@aisle.com

f3